

DESAFIO PRÁTICO | CICLO WHILE

Rastreo, contadores, validação e sentinelas

Tempo sugerido: 40 minutos | Sem executar código | Justifica as respostas

Nome: _____ Turma: _____ Data: _____

1 RASTREIA O CICLO

```
n = 2
while n <= 6:
    print(n)
    n = n + 2
print(n)
```

- a) Escreve a saída completa.
- _____
- _____
- _____
- b) Quantas iterações e quantos testes da condição ocorrem?
- _____

2 CORRIGE A REPETIÇÃO

```
n = 3
while n > 0:
    print(n)
```

- a) Explica por que este ciclo não termina.
- _____
- _____
- b) Acrescenta uma instrução para obter a contagem 3, 2, 1. Indica a sua posição.
- _____
- _____
- c) Qual é o valor final de n?
- _____

3 CONTAR E ACUMULAR

```
n = 1
total = 0
while n <= 4:
    total = total + n
    n = n + 1
```

- a) Regista total após cada iteração.
- _____
- _____
- b) Indica os valores finais de n e total.
- _____
- c) O que acontece se n começar em 5?
- _____
- _____

4 VALIDAR UMA ENTRADA

- Lê uma nota com float. Enquanto estiver fora de **[0, 20]**, pede uma nova nota. Assume que todas as entradas são numéricas.
- a) Escreve o código.
- _____
- _____
- _____
- _____
- b) Com as entradas 25, -2 e 15, quantas leituras ocorrem e qual é a nota aceite?
- _____
- _____

5 VALOR SENTINELA

- Um programa lê valores não negativos até receber -1. A sentinela não entra na soma nem na contagem.
- a) Para 5, 0, 3, -1, indica total e quantidade.
- _____
- b) Se a primeira entrada for -1, quais são os resultados?
- _____
- c) Por que zero não seria uma boa sentinela neste problema?
- _____
- _____

6 DESAFIO INTEGRADOR

- Lê vendas não negativas até -1. Calcula total, quantidade e média. Se não houver vendas, mostra **Sem vendas**. Assume entradas numéricas válidas.
- a) Escreve o programa.
- _____
- _____
- _____
- _____
- b) Testa 10, 20, -1 e, separadamente, apenas -1.
- _____
- _____

DESAFIO PRÁTICO | CICLO WHILE

Rastreo, contadores, validação e sentinelas

SOLUÇÕES

Compara os resultados e verifica o raciocínio.

1 RASTREIA O CICLO

```
2
4
6
8
```

O corpo executa **3 vezes**. A condição é testada **4 vezes**, com n igual a 2, 4, 6 e 8.

O último print está fora do ciclo. Mostra o valor de n que fez a condição tornar-se falsa: 8.

2 CORRIGE A REPETIÇÃO

a) n mantém o valor 3. A condição $n > 0$ continua verdadeira.

```
n = 3
while n > 0:
    print(n)
    n = n - 1
```

b) A atualização fica dentro do ciclo, depois do print.

c) O valor final é **0**. Não é impresso pelo corpo, porque $0 > 0$ é False.

3 CONTAR E ACUMULAR

a) Totais sucessivos: **1, 3, 6, 10**.

b) No final: **n = 5** e **total = 10**.

c) Se n começar em 5, o teste inicial é falso. O corpo executa zero vezes. Ficam **n = 5** e **total = 0**.

n controla a repetição; total acumula os valores.

4 VALIDAR UMA ENTRADA

```
nota = float(input('Nota: '))
while nota < 0 or nota > 20:
    nota = float(input('Nova nota: '))
print('Aceite:', nota)
```

b) Ocorrem **3 leituras**. A nota aceite é **15.0**.

A primeira leitura acontece antes do ciclo. Cada valor inválido provoca outra leitura dentro do ciclo.

5 VALOR SENTINELA

a) Total = **8**; quantidade = **3**. O zero é um dado válido e conta.

b) Total = **0**; quantidade = **0**. O ciclo não processa nenhum valor.

c) Zero pertence ao conjunto de valores válidos. Usá-lo para terminar impediria o registo de um valor zero.

A sentinela deve distinguir-se dos dados a processar.

6 DESAFIO INTEGRADOR

```
total = 0.0
q = 0
v = float(input('Venda: '))
while v != -1:
    total += v
    q += 1
    v = float(input('Venda: '))
print('Total:', total, 'Qtd:', q)
if q > 0:
    print('Média:', total / q)
else:
    print('Sem vendas')
```

10, 20, -1: total 30.0, quantidade 2, média 15.0.

Apenas -1: total 0.0, quantidade 0, Sem vendas.